Optimiser le traitement d'un signal quelconque.

Lucas Da Dalt

Plan

Problématique

Comment traiter de manière optimale tout signal à l'aide de la transformée de Fourier ?

- Définir l'espace fonctionnel adapté
- Présenter l'origine et la structure de la transformation discrète de Fourier
- Optimisations algorithmiques et informatiques
- Applications au filtrage et à la compression

Espace $\ell^2(\mathbb{Z}_n)$

- $\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ muni de l'addition modulaire
- Espace vectoriel de dimension n sur $\mathbb{C}$
- Produit scalaire :

$$\langle f, g \rangle = \sum_{k=0}^{n-1} f(k)\overline{g(k)}$$

Base orthogonale des exponentielles

Définition des vecteurs de base

Pour $m = 0, \ldots, n-1$,

$$e_m(k) = e^{2\pi i \frac{m\,k}{n}}, \quad k \in \mathbb{Z}_n.$$

- $\langle e_m, e_{m'} \rangle = 0$ si $m \neq m'$
- Norme $\|e_m\| = \sqrt{n}$
- Base orthogonale de $\ell^2(\mathbb{Z}_n)$

Transformée discrète de Fourier

Projection sur la base orthogonale

Pour $x \in \ell^2(\mathbb{Z}_n)$,

$$X(m) = \frac{1}{n}\langle x, e_m \rangle = \frac{1}{n}\sum_{k=0}^{n-1} x(k) e^{-2\pi i \frac{m\,k}{n}}.$$

- Rend compte de la composante fréquentielle de rang m
- Transformation bijective (inverse similaire)

Orthogonalité de la base (1/2)

On calcule

$$\langle e_m, e_{m'} \rangle = \sum_{k=0}^{n-1} e_m(k)\overline{e_{m'}(k)} = \sum_{k=0}^{n-1} e^{2\pi i \frac{(m-m')\,k}{n}}.$$

Notons $r = e^{2\pi i \frac{m-m'}{n}}$. Cette somme est une série géométrique :

$$\sum_{k=0}^{n-1} r^k = \begin{cases} n, & \text{si } r = 1, \\ \dfrac{1-r^n}{1-r}, & \text{sinon.} \end{cases}$$

Orthogonalité de la base (2/2)

Or $r = 1 \iff m - m' \equiv 0 \pmod{n}$. Dans ce cas :

$$\sum_{k=0}^{n-1} r^k = n.$$

Sinon, $r^n = 1$ mais $r \neq 1$, donc

$$\sum_{k=0}^{n-1} r^k = 0.$$

On en déduit :

$$\langle e_m, e_{m'} \rangle = \begin{cases} n, & m = m', \\ 0, & m \neq m'. \end{cases}$$

Décomposition et transformée inverse

Grâce à la base $(e_m)_{m=0..n-1}$, tout vecteur $x \in \ell^2(\mathbb{Z}_n)$ s'écrit :

$$x(k) = \frac{1}{n} \sum_{m=0}^{n-1} X(m) e_m(k), \quad \text{où} \quad X(m) = \sum_{j=0}^{n-1} x(j) e^{-2\pi i \frac{mj}{n}}.$$

Ceci constitue la transformée inverse de Fourier, permettant de passer des coefficients $X(m)$ au signal $x(k)$.

Passage du continu au discret (1/2)

Transformée continue sur $[a, b]$

$$\widehat{s}(f) = \int_a^b s(t)\, e^{-2\pi i f t}\, \mathrm{d}t.$$

Approximation par somme de Riemann

$$\widehat{s}(k) \approx \sum_{j=0}^{N-1} s\left(t_j\right) e^{-2\pi i k t_j}\, \Delta t,$$

où

$$t_j = a + j\,\Delta t, \quad \Delta t = \frac{b-a}{N}.$$

Et lorsque $N \to +\infty$, cette somme tend vers l'intégrale.

Passage du continu au discret (2/2)

Posons

$$u_j = s\left(a + j\,\tfrac{b-a}{N}\right), \quad j = 0, \ldots, N-1.$$

Alors

$$\widehat{s}(k) \approx \Delta t \sum_{j=0}^{N-1} u_j\, e^{-2\pi i k\,(a + j\,\frac{b-a}{N})}.$$

En choisissant $a = 0$ et $\Delta t = 1/N$, on retombe sur la DFT :

$$U_k = \frac{1}{N} \sum_{j=0}^{N-1} u_j\, e^{-2\pi i \frac{kj}{N}}.$$

Algorithme naïf de la DFT (1/2)

```
// Definition d un nombre complexe
typedef struct {
    double re;
    double im;
} Complexe;

/*
 * Entree :
 *    - echantillons : tableau u_j, j=0..N-1
 *    - N            : nombre de points
 * Sortie :
 *    - coeffs       : tableau U_k, k=0..N-1
 */
Complexe* calculer_dft(const Complexe *echantillons,
    size_t N) {
    Complexe *coeffs = malloc(N * sizeof(Complexe));
    for (size_t k = 0; k < N; ++k) {
        double somme_re = 0.0;
        double somme_im = 0.0;
```

Algorithme naïf de la DFT (2/2)

```
        for (size_t j = 0; j < N; ++j) {
            double angle = -2.0 * M_PI * k * j / (
                double)N;
            somme_re += echantillons[j].re * cos(angle
                )
                        - echantillons[j].im * sin(
                            angle);
            somme_im += echantillons[j].re * sin(angle
                )
                        + echantillons[j].im * cos(
                            angle);
        }
        coeffs[k].re = somme_re;
        coeffs[k].im = somme_im;
    }
    return coeffs;
}
```

Séparation des modes pairs et impairs

Soit $N = 2M$ avec N puissance de 2, et $\{u_j\}_{j=0}^{N-1}$ nos échantillons. On définit :

$$u_j^{(0)} = u_{2j}, \quad u_j^{(1)} = u_{2j+1}, \quad j = 0, 1, \ldots, M-1.$$

Alors la DFT de $\{u_j\}$ se décompose pour $0 \leq k < M$ en

$$U_k = \sum_{j=0}^{N-1} u_j \, \omega_N^{kj} = \sum_{j=0}^{M-1} u_j^{(0)} \, \omega_M^{kj} \; + \; \omega_N^k \sum_{j=0}^{M-1} u_j^{(1)} \, \omega_M^{kj},$$

et de même

$$U_{k+M} = \sum_{j=0}^{M-1} u_j^{(0)} \, \omega_M^{kj} \; - \; \omega_N^k \sum_{j=0}^{M-1} u_j^{(1)} \, \omega_M^{kj},$$

où $\omega_N = e^{-2\pi i/N}$. Ainsi, pour calculer $\{U_k\}_{k=0}^{N-1}$, il suffit de deux DFT de taille M, plus N multiplications par les « papillons » ω_N^k

Analyse de complexité et récurrence

À chaque étape sur un signal de longueur $N = 2M$, on effectue :

- deux appels récursifs à la FFT de taille M : coût $2\,T(M)$,
- N opérations de « papillons » : coût $\Theta(N)$.

La relation de récurrence s'écrit

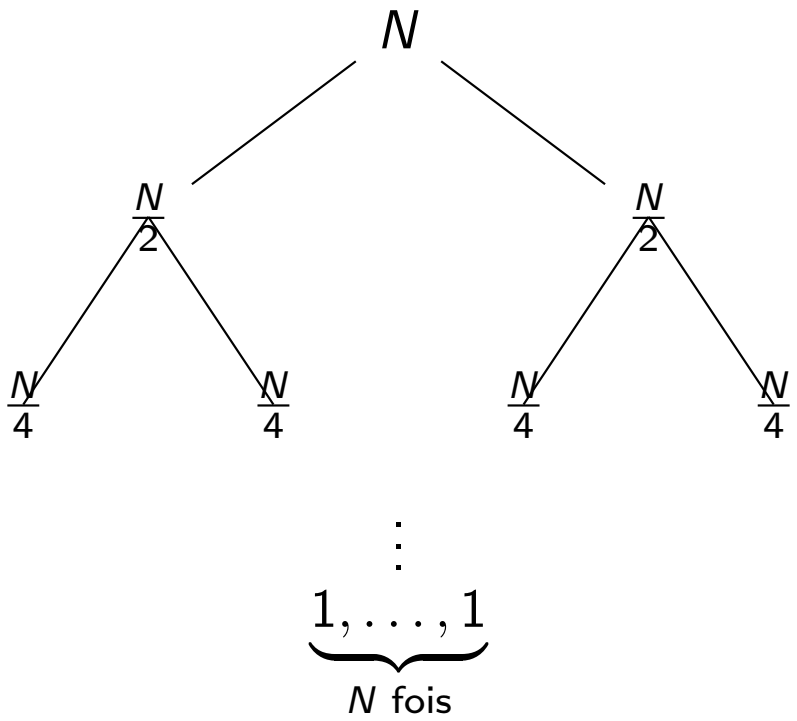
$$T(N) \;=\; 2\,T\big(\tfrac{N}{2}\big) + c\,N, \quad T(1) = \Theta(1).$$

Par application du Théorème des récurrences diviser–pour–régner, on obtient

$$T(N) = \Theta\big(N \log_2 N\big).$$

Pour $N = 2^{10} \approx 10^3$, on passe ainsi d'environ 10^6 à 10^4 opérations.

Schéma divide–and–conquer et coût total



- **Nombre de niveaux** : $\log_2 N$.
- **Coût par niveau** : chaque niveau effectue au total N opérations de papillons.

$$\text{Coût total} = \underbrace{N + N + \cdots + N}_{\log_2 N \text{ fois}} = N \log_2 N.$$

FFT récursive en $\mathcal{O}(N \log N)$ (1/2)

```
// Entree : u : vecteur de taille n, puissance de 2
// Sortie : fft u
let rec fft u =
  let n = Array.length u in
  if n = 1 then u
  else
    let m = n / 2 in
    let even = Array.init m (fun j -> u.(2*j)) in
    let odd  = Array.init m (fun j -> u.(2*j+1)) in
    let fft_even = fft even in
    let fft_odd  = fft odd in
    combine fft_even fft_odd n
```

FFT récursive en $\mathcal{O}(N \log N)$ (2/2)

```
and combine fft_even fft_odd n =
  let result = Array.make n Complex.zero in
  for k = 0 to (n/2 - 1) do
    let angle  = -2.0 *. Float.pi *. float k /. float
        n in
    let factor = Complex.expi angle in
    let tmp    = Complex.mul factor fft_odd.(k) in
    result.(k)       <- Complex.add fft_even.(k) tmp;
    result.(k + n/2) <- Complex.sub fft_even.(k) tmp
  done;
  result
```

Échantillonnage temporel et fréquentiel

Définition des paramètres :

- a : largeur de la fenêtre temporelle, intervalle $\left[-\frac{a}{2}, \frac{a}{2}\right]$.
- b : largeur de la fenêtre fréquentielle, intervalle $\left[-\frac{b}{2}, \frac{b}{2}\right]$.
- Nombre de points d'échantillonnage : N.

Points d'échantillonnage :

$$t_j = -\frac{a}{2} + j\,\frac{a}{N}, \quad u_j = s(t_j), \quad j = 0, \ldots, N-1.$$

$$f_k = -\frac{b}{2} + k\,\frac{b}{N}, \quad k = 0, \ldots, N-1.$$

On approxime alors l'intégrale de Fourier par

$$\widehat{s}(f_k) \approx \sum_{j=0}^{N-1} u_j \, \exp\bigl(-i\, t_j\, f_k\bigr)\, \frac{a}{N}.$$

Condition $ab = 2\pi N$ et DFT classique

Pour que $t_j\, f_k = 2\pi \dfrac{j\,k}{N} + \text{constante}$, on impose la relation

$$a\,b \;=\; 2\pi\, N.$$

En effet, avec t_j, f_k ci-dessus :

$$t_j\, f_k = \left(-\tfrac{a}{2} + j\,\tfrac{a}{N}\right)\left(-\tfrac{b}{2} + k\,\tfrac{b}{N}\right) = 2\pi\,\frac{j\,k}{N} \;+\; \text{phase indépendante de } j, k.$$

D'où, en extrayant cette phase globale,

$$\widehat{s}(f_k) \approx \frac{a}{N}\, e^{-i\,\text{phase}} \sum_{j=0}^{N-1} u_j\, e^{-2\pi i\,\frac{j\,k}{N}} \;\propto\; \frac{1}{N}\sum_{j=0}^{N-1} u_j\, e^{-2\pi i\,\frac{j\,k}{N}} = U_k.$$

$$\boxed{U_k = \frac{1}{N}\sum_{j=0}^{N-1} u_j\, e^{-2\pi i\,\frac{j\,k}{N}}}$$

Convolution discrète et transformée

Soient deux signaux échantillonnés $u, v \in \ell^2(_N)$.

$$(u * v)_n = \sum_{j=0}^{N-1} u_j \, v_{n-j \bmod N}, \quad n = 0, \ldots, N-1.$$

Leur DFT « normalisée » est

$$U_k = \sum_{j=0}^{N-1} u_j \, \omega_N^{jk}, \quad V_k = \sum_{j=0}^{N-1} v_j \, \omega_N^{jk}, \quad \omega_N = e^{-2\pi i/N}.$$

Alors on montre par un échange de sommes :

$$\mathrm{DFT}\big(u * v\big)_k = \sum_{n=0}^{N-1} \Big(\sum_{j=0}^{N-1} u_j \, v_{n-j} \Big) \omega_N^{nk} = U_k \, V_k.$$

C'est le théorème de convolution :

$$\widehat{u * v} \;=\; \widehat{u} \;\times\; \widehat{v}.$$

Calcul rapide de la convolution

1. Calculer $U = \mathrm{FFT}(u)$ et $V = \mathrm{FFT}(v)$ en $\mathcal{O}(N \log N)$.
2. Former le produit point-à-point $W_k = U_k V_k$.
3. Appliquer la FFT inverse à W pour obtenir $u * v$.

Grâce à la propriété de convolution, le coût total est

$$2\,T(N) + \Theta(N) \;=\; 2\big(\Theta(N \log N)\big) + \Theta(N) \;=\; \Theta(N \log N).$$

Ainsi la convolution circulaire se calcule en $\mathcal{O}(N \log N)$.

Étape 1 :identite remarquable

On part de la TFFr discrète d'ordre α sur $\{u_j\}$ $(j = 0, \ldots, N-1)$:

$$U_k^{(\alpha)} = e^{-i\frac{\alpha\pi}{4}} \sum_{j=0}^{N-1} u_j \, e^{-i\frac{\alpha\pi}{N}j^2} \, e^{i\frac{2\pi jk}{N}} \, e^{-i\frac{\alpha\pi}{N}k^2}.$$

On réécrit cette somme en isolant $e^{i\frac{2\pi jk}{N}}$:

$$U_k^{(\alpha)} = e^{-i\frac{\alpha\pi}{4}} \, e^{-i\frac{\alpha\pi}{N}k^2} \sum_{j=0}^{N-1} \underbrace{u_j \, e^{-i\frac{\alpha\pi}{N}j^2}}_{x_j} \times \underbrace{e^{i\frac{2\pi jk}{N}}}_{\omega_N^{jk}}.$$

En notant $x_j = u_j \, e^{-i\frac{\alpha\pi}{N}j^2}$, la somme devient un produit scalaire $\sum_j x_j \, \omega_N^{jk}$, c'est-à-dire la DFT de $\{x_j\}$.

Étape 2 : convolution et FFT

Le terme $e^{i\frac{2\pi jk}{N}}$ peut se voir comme la convolution circulaire de deux suites:

$$\omega_N^{jk} = \sum_{\ell=0}^{N-1} y_\ell \, z_{k-\ell},$$

avec

$$y_\ell = e^{i\frac{\alpha\pi}{N}\ell^2}, \quad z_m = e^{i\frac{\alpha\pi}{N}m^2}.$$

$$\sum_{j=0}^{N-1} x_j \, \omega_N^{jk} = (x * y)_k,$$

et par le théorème de convolution :

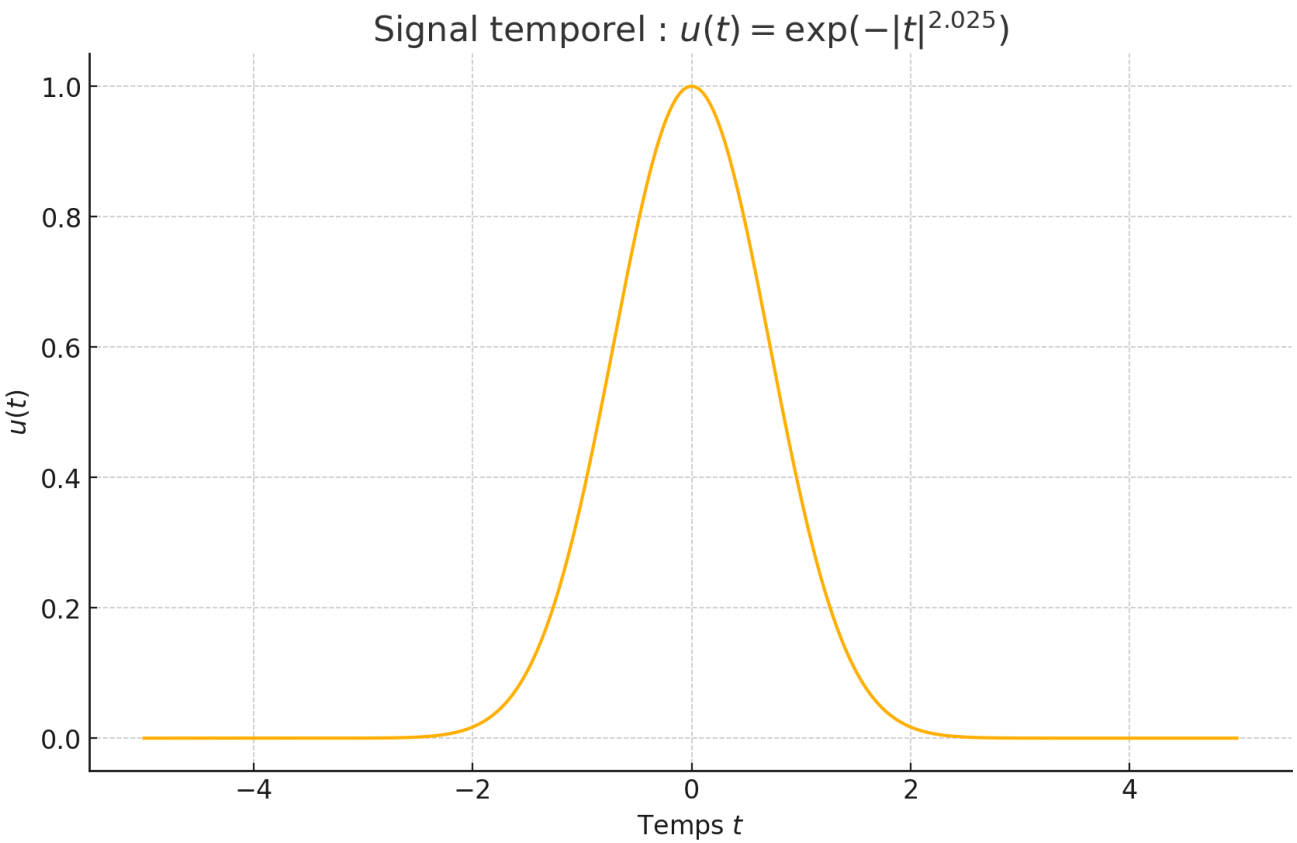
$$\mathrm{DFT}(x * y) = \mathrm{DFT}(x) \times \mathrm{DFT}(y).$$

D'où l'algorithme FFT pour la TFFr en $\mathcal{O}(N \log N)$:

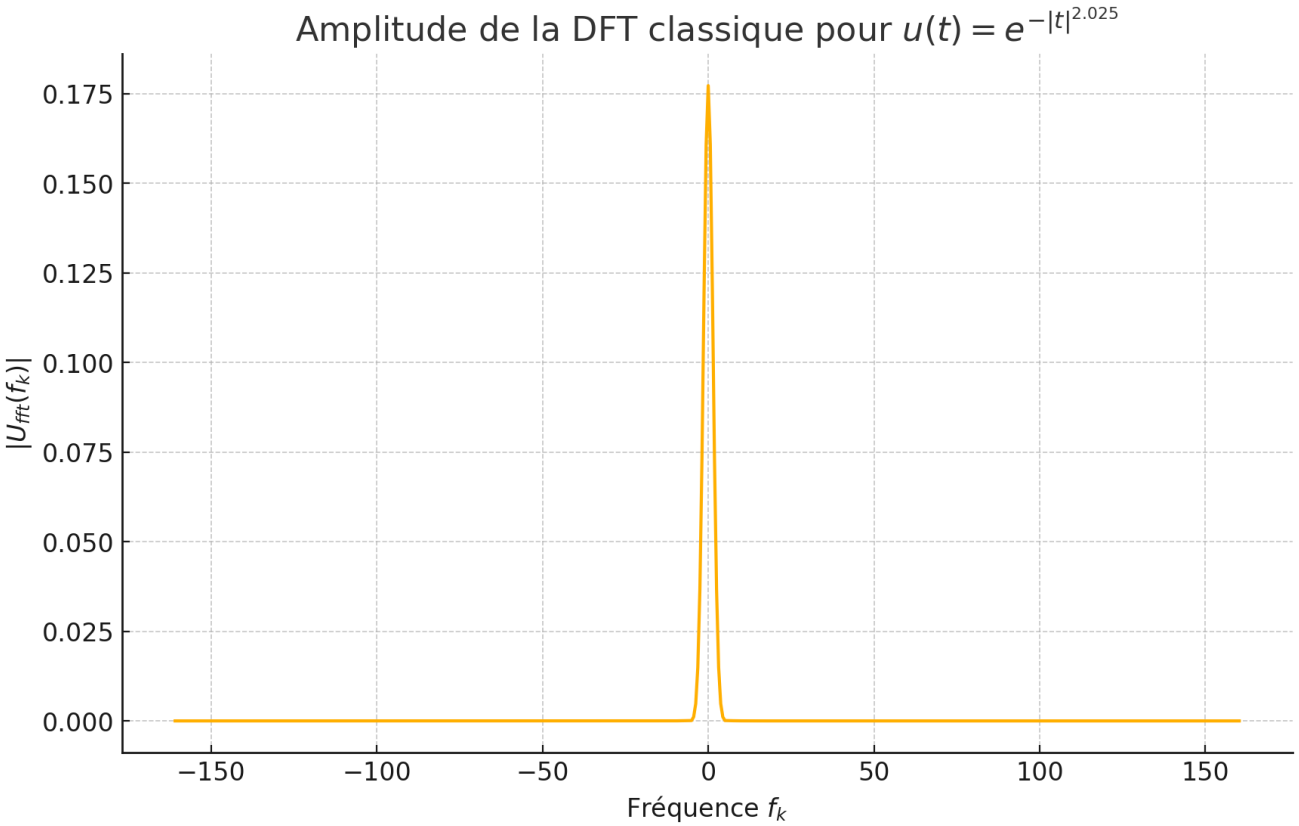
$$x_j = u_j \, e^{-i\frac{\alpha\pi}{N}j^2}, \quad y_j = e^{i\frac{\alpha\pi}{N}j^2},$$

puis on calcule FFT(x), FFT(y), on multiplie point à point, et enfin on remet le facteur $e^{-i\frac{\alpha\pi}{4}} \, e^{-i\frac{\alpha\pi}{N}k^2}$.

Signal



DFT


Amplitude de la DFT classique pour $u(t) = e^{-|t|^{2.025}}$
$|U_{fft}(f_k)|$
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
−150
−100
−50
0
50
100
150
Fréquence f_k

Questions ?

Merci pour votre attention